```
Authorization: Bearer u9wu69dl5L8AT9FNe03TM-s4H8M #A
Authorization: Basic ZGVtbzpwYXNzd29yZA== #B
```

#A The capability token.
#B The user authentication credentials.

This is a violation of the HTTP specification, which allows only a single Authorization header, and the Natter API (and many frameworks) will only see the first header value and so the user authentication will fail. The solution to this problem is to use a different header for one of the credentials. For Natter, you'll switch back to using cookies for authentication and revert to using the custom `X-CSRF-Token` header to hold the anti-CSRF token.

> **WARNING** You may wonder if you can do away with the anti-CSRF token now that you are using capabilities for access control, which are immune to CSRF. This would be a mistake, because an attacker that has a genuine capability to access the API can still use a CSRF to make their requests appear to come from a different user. You have swapped ambient authority for *ambient identity*, which can still be problematic. One way to view an anti-CSRF token is as a capability that grants permission to use the associated cookie.

To switch back to using cookies for authentication, open the Main.java file in your editor and find the lines that create the `TokenController` object. Change the `tokenStore` variable to use the `CookieTokenStore` that you developed back in chapter 4: